Assignment 1

What Happens When I Visit the Google Homepage from My Mobile Browser?

1. Introduction

When a user opens a web browser on a mobile phone and enters www.google.com in the address bar, the Google homepage appears on the screen after a short period of time. From the user's point of view, this looks like a simple operation. However, several processes take place between entering the website address and seeing the final webpage.

The browser, mobile operating system, local network, DNS servers, Internet service provider, Internet routers, Google's infrastructure, web servers, and the browser's rendering engine all participate in this process.

The process can be divided into four major stages:

1. Client-side operations involved in creating and sending the request.
2. Network operations that carry the request from the mobile device to Google's infrastructure.
3. Server-side operations performed after Google receives the request.
4. Client-side operations that process the response and display the webpage.

2. Client-Side Operations — Request Handling

Client-side request handling begins when the user interacts with the mobile browser. Several operations occur before the request reaches Google's servers.

2.1 Opening the Chrome Browser

The user first opens Google Chrome on the mobile phone. The operating system starts or resumes the Chrome application and provides it with access to the device's network connection.Chrome creates the necessary browser processes and prepares the interface for accepting the user's input.

2.2 Entering the Website Address

The user types:

www.google.com

into the address bar and presses Enter.

Chrome identifies that the entered text represents a website address rather than a search query.

The browser interprets the address and determines the protocol and destination that should be used to access the website.

2.3 URL Processing

The browser breaks the address into different components.

For example:

- Protocol: HTTPS
- Host/domain: www.google.com
- Port: HTTPS normally uses port 443
- Path: / when the root page is requested

The browser therefore needs to communicate securely with Google's web infrastructure.

2.4 Checking Browser Information

Before making a new network request, the browser can check information that may already be available locally, such as cached resources, DNS information, or existing connection information.

The user is assumed to be visiting the website for the first time on that device, so the browser may need to perform several operations that would not necessarily be required on a subsequent visit.

2.5 DNS Resolution

The browser needs an IP address to communicate with the destination server. A domain name such as www.google.com is easier for humans to remember, while network communication ultimately uses IP addresses.

The device therefore performs DNS resolution.

A simplified sequence is:

www.google.com → DNS lookup → IP address

The browser or operating system may first check locally available DNS information. If the required information is not available, a DNS request is sent to the configured DNS resolver.

The DNS system finds the appropriate records and returns an IP address that can be used to reach Google's infrastructure.

2.6 Preparing the Network Connection

After obtaining an appropriate destination address, Chrome needs to establish communication with Google's servers.For a traditional HTTPS connection, this involves establishing a transport connection and then securing it with TLS.

Depending on the networking protocol and configuration used by the browser and server, modern browsers may also use HTTP/3 over QUIC rather than HTTP/2 over TCP. The exact protocol can vary depending on the connection and server configuration.

2.7 Establishing a Secure Connection

The browser establishes a secure communication channel.

With a TCP-based HTTPS connection, the process broadly involves:

1. Establishing a TCP connection.
2. Starting the TLS handshake.
3. Negotiating cryptographic parameters.
4. Verifying the server certificate.
5. Establishing encryption keys.
6. Creating an encrypted communication channel.

This prevents the contents of the communication from being transmitted as ordinary readable data across the network.

2.8 Creating the HTTP Request

Once the secure connection is ready, Chrome creates an HTTP request for the webpage.

Conceptually, the request contains information such as:

- HTTP method
- Requested resource
- Host information
- Browser information
- Accepted content types
- Language preferences
- Connection-related information
- Other HTTP headers

The browser then sends this request through the network connection.

3. Network Operations

Network operations describe what happens between the mobile device and Google's infrastructure.The request does not normally travel directly from the phone to one physical Google server. It passes through several networking components and systems.

3.1 Mobile Network or Wi-Fi

The mobile phone first communicates with its local network.If the phone is connected through Wi-Fi, the data travels through:

Mobile phone → Wi-Fi access point/router → ISP

If mobile data is being used, the communication goes through the mobile carrier's network before reaching the wider Internet.

3.2 Local Network Routing

The device sends packets toward its network gateway.The router or gateway determines where the packets should be forwarded.The data is divided into network packets so that it can travel through the network infrastructure.

3.3 Internet Service Provider

The packets reach the user's Internet service provider.The ISP provides connectivity between the user's local network and other networks on the Internet.The ISP's routers examine the destination information and forward the packets toward the appropriate external network.

3.4 Internet Routing

The request may pass through multiple routers and autonomous systems before reaching Google's network.Routers use routing information to determine suitable paths for forwarding packets.The actual physical route can vary depending on network conditions, routing policies, connectivity, and other factors.

A simplified representation is:

Phone → Router → ISP → Internet Routers → Google's Network

3.5 Transport Protocol

For a TCP-based connection, TCP provides reliable delivery of data between the client and server.

TCP helps with:

- Establishing a connection
- Ordering data
- Detecting lost packets
- Retransmitting missing data
- Controlling the flow of information
- Managing congestion

If HTTP/3 is used instead, QUIC provides transport functionality over UDP.

3.6 TLS Encryption

Because Google is accessed using HTTPS, the communication is protected using TLS.

TLS helps provide:

- Confidentiality
- Integrity
- Server authentication

This means that the communication between the browser and the Google service is protected against unauthorized modification or straightforward interception.

3.7 Reaching Google's Network

Eventually, the request reaches Google's network.

Google operates a large distributed infrastructure rather than relying on one single physical machine for all requests.

The request can therefore be handled by an appropriate part of Google's infrastructure based on factors such as network location, availability, capacity, and service configuration.

4. Server-Side Operations

Once the request reaches Google's infrastructure, processing takes place on the server side.The exact internal architecture of Google's systems is not publicly exposed in complete detail, so the following describes the general high-level operations involved in serving a modern web request.

4.1 Receiving the Request

Google's infrastructure receives the HTTPS request sent by the browser.The request contains information that allows the infrastructure to determine which service and resource the client is requesting.In this case, the requested website is Google's homepage.

4.2 Handling Incoming Traffic

Large web services generally use infrastructure capable of handling a very large number of requests.

Incoming traffic can be handled by front-end systems and load-balancing mechanisms that distribute requests across available resources.

This helps prevent one individual server from becoming a bottleneck.

4.3 Request Routing

The request is routed to the appropriate service or server responsible for handling the requested content.

The infrastructure may consider factors such as:

- Requested hostname
- Network location
- Server availability
- Current load
- Service configuration
- Requested resource

4.4 Processing the Request

The appropriate server-side software processes the request.It determines what content or response should be returned to the browser.For a basic homepage request, the response can include the HTML required to begin displaying the page.

4.5 Generating or Retrieving Content

The server-side infrastructure prepares the requested webpage and associated information.Depending on the particular resource, content may be generated dynamically, retrieved from internal services, or served from cached/distributed infrastructure.A modern webpage is normally made up of more than a single HTML document. The initial HTML can reference additional resources that the browser may request separately.

4.6 Preparing the HTTP Response

The server creates an HTTP response.

The response generally contains:

- HTTP status information
- Response headers

- Content
- Other metadata

For a successful request, an appropriate success status is returned.The response is then transmitted back toward the user's device over the secure connection.

4.7 Sending the Response

The response travels from Google's infrastructure through the Internet back toward the user's mobile device.The network may use a different physical or logical path for the return traffic.The packets eventually reach the user's network and then the mobile phone.

5. Client-Side Operations — Response Handling

After Google's response reaches the mobile device, Chrome has to process the received information and convert it into the webpage visible to the user.This stage involves several browser operations.

5.1 Receiving the Response

Chrome receives the response from Google's server.Because the connection is protected using HTTPS, the browser processes the encrypted communication and obtains the response data through the established secure connection.

5.2 Processing HTTP Response Information

The browser examines the response status and headers.The headers provide information about how the browser should handle the returned content.Chrome then begins processing the response body.

5.3 Parsing HTML

The browser receives HTML and begins parsing it.The HTML parser converts the document structure into a Document Object Model, commonly called the DOM.The DOM represents the elements of the webpage in a structure that the browser can work with.For example, elements such as:

- HTML
- Head
- Body
- Divisions
- Text
- Buttons
- Images
- Links can become part of the DOM structure.

5.4 Discovering Additional Resources

While processing HTML, Chrome may discover references to additional resources.

These can include:

- CSS files
- JavaScript files
- Images
- Fonts
- Other webpage resources

The browser may make additional network requests to retrieve these resources.Therefore, loading a webpage can involve multiple requests rather than just one request.

5.5 Processing CSS

CSS determines how elements should appear on the screen.The browser parses CSS information and creates the structures required to determine the visual presentation of the webpage.The browser combines information from the HTML and CSS to determine which styles apply to different elements.

5.6 JavaScript Processing

If JavaScript is included in the webpage, Chrome's JavaScript engine can execute it.

JavaScript can:

- Modify webpage content
- Change styles
- Respond to user interactions
- Request additional data
- Update elements dynamically

The exact JavaScript operations depend on the content delivered by the website.

5.7 Creating the Render Structure

The browser determines which webpage elements need to be displayed and how they should be styled.The relevant information is combined into structures used by the rendering process.

5.8 Layout Calculation

Chrome calculates the size and position of visible elements.

This includes determining:

- Width
- Height
- Position
- Spacing
- Alignment
- Relationships between elements

This stage is commonly referred to as layout.

5.9 Painting

After determining the layout, the browser prepares the visual representation of the page.Text, backgrounds, borders, images, and other visual elements are painted.

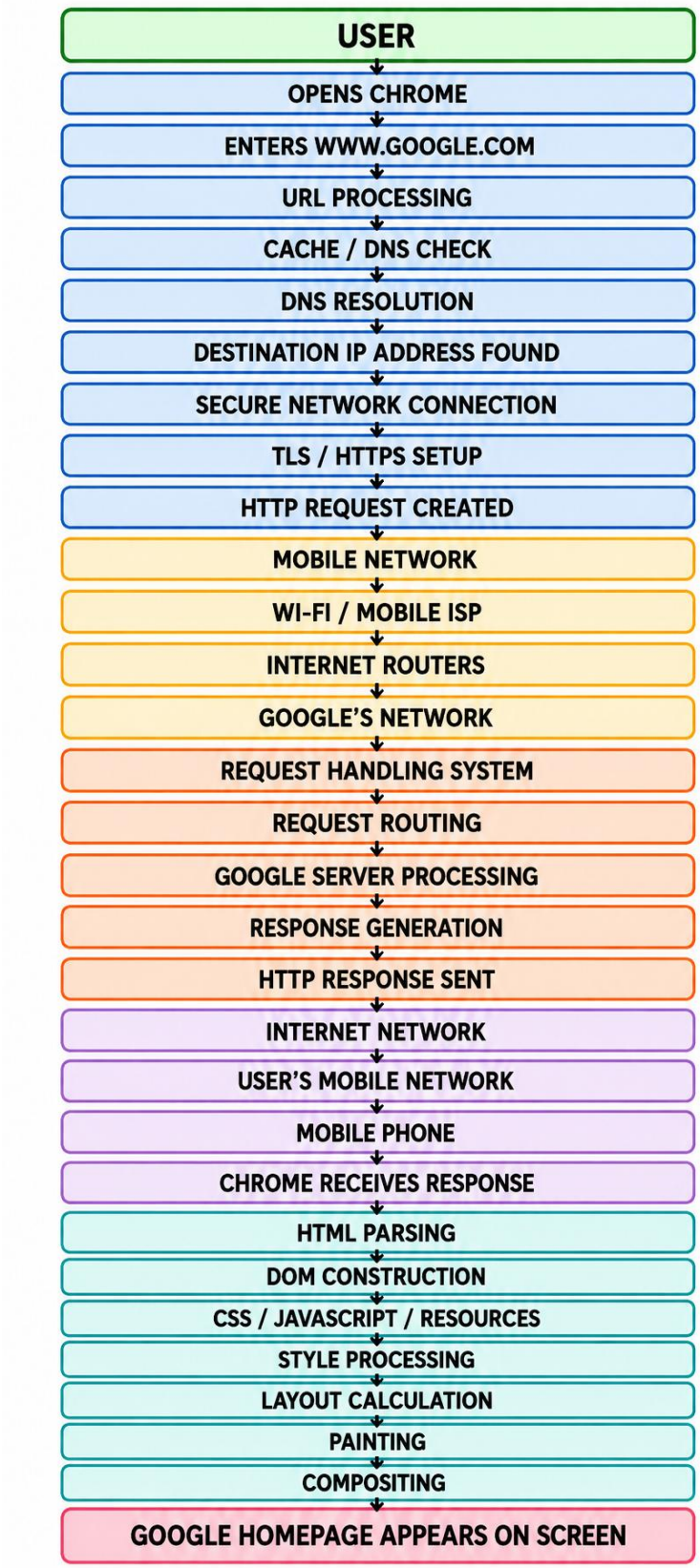
5.10 Compositing

The browser can divide the rendered page into different layers and combine those layers to produce the final visual output.This helps the browser efficiently display the webpage on the device's screen.

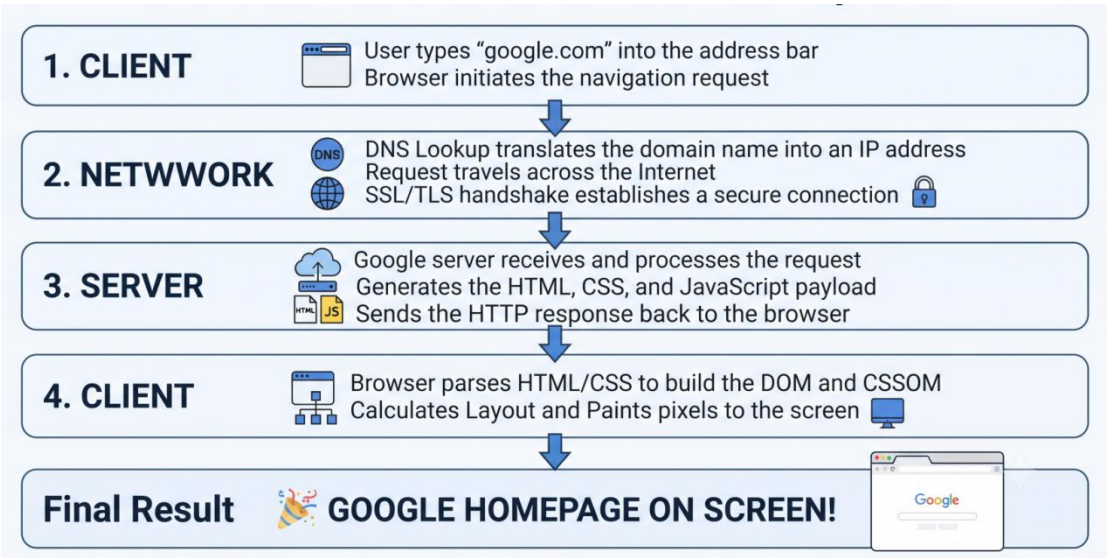
5.11 Displaying the Google Homepage

Finally, the rendered content is presented on the mobile screen.The user sees the Google homepage and can now interact with it.The complete process that appeared to take only a few seconds actually involved many operations between the initial URL entry and the final display.

6. Flow Diagram



Simplified Four-Stage View



7. Conclusion

Visiting www.google.com may look like a very simple action to the user, but a large number of operations take place in the background.

The process starts when the user enters the website address in Chrome. The browser interprets the URL, performs the required DNS resolution, establishes secure communication, and creates a request. The request then travels through the local network, Internet service provider, Internet routers, and Google's network.

After receiving the request, Google's infrastructure processes it and prepares an appropriate response. The response then travels back through the network to the mobile device.Chrome receives the response and begins processing the returned content. It parses HTML, processes CSS and JavaScript, requests additional resources when required, calculates the page layout, paints the visual elements, and finally displays the webpage.

Therefore, the appearance of a webpage is the result of cooperation between multiple components:

User → Browser → DNS → Network → Google's Infrastructure → Server → Network → Browser → Rendering Engine → Screen

This demonstrates that a modern webpage depends on several layers of technology working together. What appears to the user as a single action—opening Google—actually involves client-side processing, network communication, server-side processing, and browser rendering.

8. Optional Part — Chrome Developer Tools

Objective

To observe how www.google.com loads and how the browser communicates with Google's servers using Chrome Developer Tools.

Experimental Setup

- **Device:** Laptop/Desktop
- **Browser:** Google Chrome Incognito
- **Tool:** Chrome DevTools → Network
- **Network:** No throttling
- **Cache:** Enabled

Procedure

1. Open Chrome Incognito.
2. Press Ctrl + Shift + I.
3. Select the Network tab.
4. Enter www.google.com.
5. Press Enter and observe the requests and Waterfall.

Observations

The experiment showed:

- 39 requests
- 18.8 kB data transferred
- 2.5 MB total resources
- 2.13 seconds finish time
- 328 ms DOM Content Loaded
- 556 ms Load time

Different resources such as HTML, JavaScript, CSS, images, fonts, and Ping requests were observed.

Sequential vs Parallel Requests

Requests were not completely sequential. The Waterfall showed overlapping requests, meaning multiple resources can be loaded concurrently.



└── Other Requests

Thus, webpage loading uses a combination of sequential and parallel operations.

Logged-In vs Non-Logged-In

The experiment was performed without logging in. A logged-in session may generate additional requests related to authentication, account information, and personalization.

Key Findings

1. Loading a webpage involves multiple network requests.
2. 39 requests were observed in this experiment.
3. Requests can execute concurrently.
4. The Waterfall shows request timing and dependencies.
5. Chrome DevTools helps understand network communication and webpage loading.

